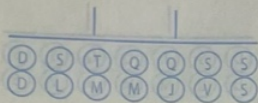


O artigo "The Psychology and Economics of Software Testing", argumenta que o teste de software não é apenas uma tarefa técnica, ele também envolve aspectos cruciais de psicologia e economia. O objetivo principal do teste de software não é provar que um programa funciona, mas encontrar o maior número possível de erros. Definir o teste como um processo para encontrar erros tem um efeito psicológico importante: se o objetivo é demonstrar que não há erros, os testadores tendem a escolher dados de teste menos propensos a causar falhas. Por outro lado, se o objetivo é encontrar erros, os testadores usarão dados que têm uma maior probabilidade de descobrir falhas.

O artigo também sugere que o teste é um processo destrutivo, o que pode ser psicologicamente difícil, já que a maioria das pessoas tem uma inclinação construtiva. A percepção de sucesso também é invertida na prática de testes. Enquanto muitos gerentes de projeto consideram "bem-sucedida" uma execução de teste que não encontra erros, o artigo afirma que um teste bem-sucedido é aquele que encontra erros que podem ser corrigidos. Um teste que não encontra erros, em muitos casos, é considerado mal-sucedido porque a ausência de erros em um programa é basicamente irrealista. Uma analogia mé-



dica é usada para ilustrar o ponto: um teste laboratorial é bem-sucedido quando diagnostica uma doença, não quando não encontra nada.

A economia do Teste de Software

É impossível encontrar todos os erros em um programa, mesmo que seja trivial. Isso se deve à inviabilidade econômica e prática de realizar testes exaustivos. O objetivo do teste de software deve ser, portanto, maximizar o número de erros encontrados com um número finito de casos de teste.

O texto discute duas estratégias de teste principais:

→ Teste de caixa-preta (Black-Box Testing): foca em encontrar erros ao usar dados de teste baseados apenas nas especificações de um programa, sem considerar sua estrutura interna.

→ Teste de caixa-branca (White Box Testing): os dados de teste são criados a partir de uma análise da lógica do programa.

O artigo conclui que, a melhor abordagem é criar uma estratégia de teste razoável, embora não infalível.